Automates Avancés et Applications

Auteur
Yago Iglesias

10 octobre 2024

Table des matières

1 Introduction **1**

2 Langages Rationnels **1**
2.1 Définitions de base 2
2.2 Langages rationnels : définitions 2

3 Automates finis **3**
3.1 Automates finis déterministes 3
3.2 Automates finis non déterministes + ε-transitions 4
3.3 Déterminisation d'un AFN 5
3.4 Équivalence entre expressions rationnelles et automates finis déterministes 6

4 Minimisation d'automates **7**
4.1 Morphismes d'automates 7
4.2 Minimisation d'automates 9
4.2.1 Algorithme de Brzozowski 10
4.2.2 Algorithme de Moore 11

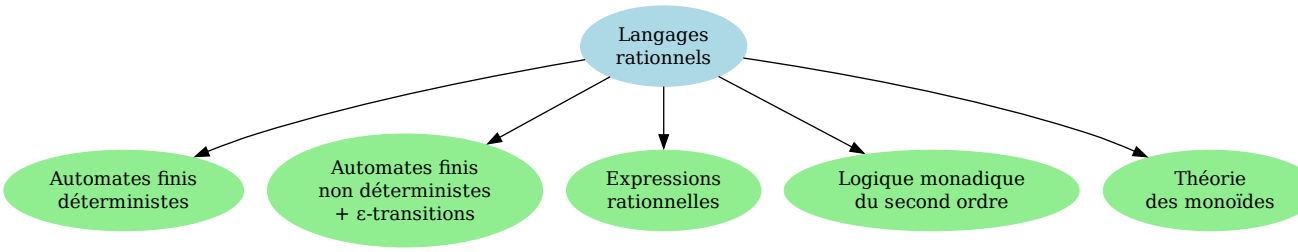
5 Monoïdes **12**

1 Introduction

Ce document est un recueil de notes du cours d'Automates Avancés et Applications de niveau M1. Il est basé sur les cours de Mme. Daniela Petrisan et M. Roberto Mantaci à Université Paris Cité, cependant toute erreur ou inexactitude est de ma responsabilité.

Ce document a été rédigé principalement par Yago Iglesias, mais tout contributeur peut être retrouvé dans la section contributeurs du répertoire GitHub. Un remerciement particulier est adressé à Erin Le Boulc'h pour sa participation active à la rédaction et correction de ce document.

2 Langages Rationnels



2.1 Définitions de base

Définition 2.1 (alphabet)**.** Un **alphabet** est un ensemble fini de lettres ou de symboles.

Définition 2.2 (mot)**.** Un **mot** sur un alphabet Σ est une séquence de lettres de Σ. On écrit $w = w_1 w_2 \dots w_n$ où $w_i \in \Sigma, \forall i \in \{1, \dots, n\}$. La longueur d'un mot w est notée $|w| = n$, pour $w = w_1 w_2 \dots w_n$.

Notation 2.1. On note ε le mot vide et Σ^* l'ensemble des mots sur Σ.

Définition 2.3 (concaténation)**.** La **concaténation** de deux mots w et v est notée wv. Si $w = w_1 w_2 \dots w_n$ et $v = v_1 v_2 \dots v_m$, alors $wv = w_1 w_2 \dots w_n v_1 v_2 \dots v_m$.

Définition 2.4 (langage)**.** Un langage sur un alphabet Σ est un sous-ensemble de Σ^*.

Définition 2.5 (concaténation de langages)**.** Soient L_1, L_2 deux langages sur Σ, leur **concaténation** est le langage

$$L_1 L_2 = \{w_1 w_2 \mid w_1 \in L_1, w_2 \in L_2\}$$

Exemple 2.1.1.

$$\Sigma = \{a, b\}, \quad L = \{a, ab, bb\}, \quad K = \{b, ab\}$$

$$LK = \{ab, abb, bbb, aab, abab, bbab\}$$

Définition 2.6 (étoile de Kleene)**.** Soit L un langage sur Σ, son **étoile de Kleene** est le langage

$$L^* = \bigcup_{n \in \mathbb{N}} L^n$$

où $L^0 = \{\varepsilon\}$ et $L^{n+1} = LL^n$.

Remarque 2.1.1. *Est-ce que $L^* = \{w^n \mid w \in L, n \in \mathbb{N}\}$?*
Non, on peut trouver un contre-exemple avec $L = \{a, b\}$.
On a bien que $ab \in L^*$, mais $ab \notin \{w^n \mid w \in L, n \in \mathbb{N}\} = \{a^n \mid n \in \mathbb{N}\} \cup \{b^n \mid n \in \mathbb{N}\}$.

Remarque 2.1.2. *Est-ce que $L(M \cap N) = LM \cap LN$?*
Non, on peut trouver un contre-exemple avec $L = \{a, ab\}$, $M = \{b\}$ et $N = \{\varepsilon\}$.
On a que $M \cap N = \emptyset$, donc $L(M \cap N) = \emptyset$.
Mais $LM = \{ab, abb\}$ et $LN = \{a, ab\}$, donc $LM \cap LN = \{ab\} \neq \emptyset$.

Exercice 2.1.1. Montrer que la concaténation de langages est distributive par rapport à l'union, *i.e.* que $L(M \cup N) = LM \cup LN, \forall L, M, N$ langages.

2.2 Langages rationnels : définitions

Définition 2.7 (langage rationnel)**.** Soit Σ un alphabet fini, l'ensemble ERat des expressions rationnelles sur Σ est défini comme suit :

- $\varepsilon \in ERat$
- $\emptyset \in ERat$
- $\forall a \in \Sigma, a \in ERat$
- $\forall E, F \in ERat, E + F \in ERat$
- $\forall E, F \in ERat, EF \in ERat$

— $\forall E \in ERat, E^* \in ERat$

Définition 2.8 (sémantique des expressions rationnelles)**.** Soit $r \in$ ERat, on définit le langage $\mathcal{L}(r)$ associé à r par induction sur la structure de r :
— $\mathcal{L}(\varepsilon) = \{\varepsilon\}$
— $\mathcal{L}(\emptyset) = \emptyset$
— $\mathcal{L}(a) = \{a\}$
— $\mathcal{L}(E + F) = \mathcal{L}(E) \cup \mathcal{L}(F)$
— $\mathcal{L}(EF) = \mathcal{L}(E)\mathcal{L}(F)$
— $\mathcal{L}(E^*) = \mathcal{L}(E)^*$

Exemple 2.2.1. $\Sigma = \{a, b\}$, $\quad r = (a + b)^* a \in$ ERat, alors $\mathcal{L}(r) = \{wa \mid w \in \Sigma^*\}$

Définition 2.9 (langage rationnel)**.** Un langage L sur un alphabet Σ est dit **rationnel** s'il existe une expression rationnelle $r \in$ ERat telle que $L = \mathcal{L}(r)$.

Exemple 2.2.2. $\{a^n \mid n \in \mathbb{N}\}$ est un langage rationnel engendré par l'expression rationnelle a^*.
Cependant, $\{a^n b^n \mid n \in \mathbb{N}\}$ n'est pas un langage rationnel.

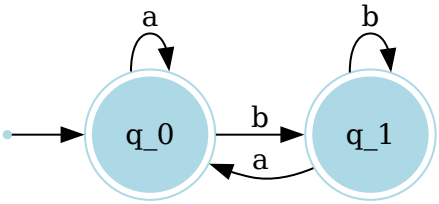
3 Automates finis

3.1 Automates finis déterministes

Définition 3.1 (automate fini déterministe)**.** Soit Σ un alphabet, un **automate fini déterministe** (AFD) est un tuple $\langle Q, q_0, F, \delta \rangle$ où
— Q est un ensemble fini d'états
— $q_0 \in Q$ est appelé l'état initial
— $F \subseteq Q$ est l'ensemble des états finaux / acceptants
— $\delta : Q \times \Sigma \to Q$ est la fonction de transition

Exemple 3.1.1. Un automate fini déterministe :

— $\Sigma = \{a, b\}$
— $Q = \{q_0, q_1\}$
— q_0 est l'état initial
— $F = \{q_0, q_1\}$
— $\delta : \begin{cases} (q_0, a) & \mapsto q_0 \\ (q_0, b) & \mapsto q_1 \\ (q_1, a) & \mapsto q_0 \\ (q_1, b) & \mapsto q_1 \end{cases}$



Définition 3.2 (lecture d'un mot par un AFD)**.** Soit $A = \langle Q, q_0, F, \delta \rangle$ un AFD qui a pour alphabet

Σ. On définit la fonction δ^* par induction sur la longueur du mot w :

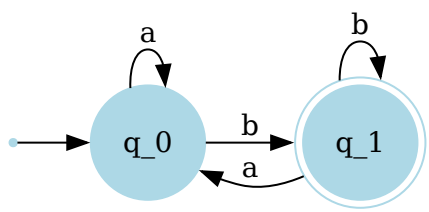
$$\begin{array}{rcl} \delta^* : Q \times \Sigma^* & \to & Q \\ (q, \varepsilon) & \mapsto & q \\ (q, wa) & \mapsto & \delta(\delta^*(q, w), a) \end{array}$$

On a alors que $\delta^*(q, w)$ est l'état atteint par A après avoir lu le mot w depuis l'état q.

Définition 3.3 (langage reconnu par un AFD)**.** Le langage reconnu / accepté par un AFD $A = \langle Q, q_0, F, \delta \rangle$ est le langage

$$\mathcal{L}(A) = \{w \mid w \in \Sigma^*, \delta^*(q_0, w) \in F\}$$

Exemple 3.1.2. Pour l'automate A suivant, avec $\Sigma = \{a, b\}$, le langage reconnu est $\mathcal{L}(A) = \mathcal{L}((a+b)^*b) = \mathcal{L}((a+b)^*bb^*) = \{wb \mid w \in \Sigma^*\}$.

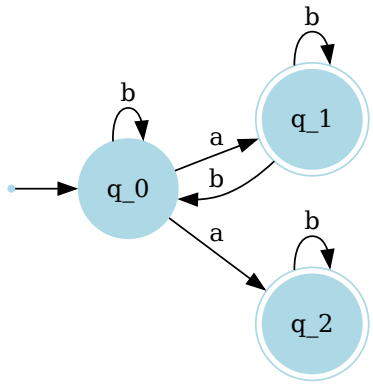


3.2 Automates finis non déterministes + ε-transitions

Définition 3.4 (automate fini non déterministe + ε-transitions)**.** Soit Σ un alphabet, un **automate fini non déterministe** (AFN) est un tuple $\langle Q, I, F, \delta \rangle$ où

- Q est un ensemble fini d'états
- $I \subseteq Q$ est l'ensemble des états initiaux
- $F \subseteq Q$ est l'ensemble des états finaux / acceptants
- $\delta : Q \times (\Sigma \cup \{\varepsilon\}) \to \mathcal{P}(Q)$ est la fonction de transition

Exemple 3.2.1. Un automate fini non déterministe avec $\Sigma = \{a, b\}$:



Définition 3.5 (lecture d'un mot par un AFN)**.** Soit $A = \langle Q, I, F, \delta \rangle$ un AFN qui a pour alphabet Σ. On définit la fonction δ^* par induction sur la longueur du mot w :

$$\begin{array}{rcl} \delta^* : Q \times \Sigma^* & \to & \mathcal{P}(Q) \\ (q, \varepsilon) & \mapsto & \{q\} \\ (q, wa) & \mapsto & \bigcup\limits_{p \in \delta^*(q,w)} \delta(p, a) \end{array}$$

Définition 3.6 (langage reconnu par un AFN)**.** Le langage reconnu / accepté par un AFN $A = \langle Q, I, F, \delta \rangle$ est le langage

$$\mathcal{L}(A) = \{w \in \Sigma^* \mid \exists q_0 \in I, \delta^*(q_0, w) \cap F \neq \emptyset\}$$

3.3 Déterminisation d'un AFN

Soit $A = \langle Q, I, F, \delta \rangle$ un AFN, on considère l'AFD $A' = \langle \mathcal{P}(Q), I, F', \delta' \rangle$ où

— $F' = \{q \in \mathcal{P}(Q) \mid q \cap F \neq \emptyset\}$

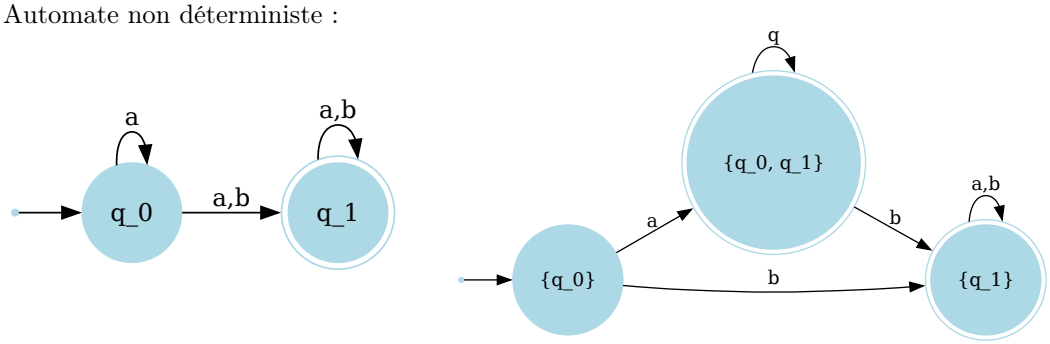
— $$\begin{array}{rcl} \delta' : \mathcal{P}(Q) \times \Sigma & \to & \mathcal{P}(Q) \\ (Q, a) & \mapsto & \bigcup\limits_{p \in Q} \delta(p, a) \end{array}$$

Ce processus est appelé **déterminisation** d'un AFN et nous permet de transformer un AFN en un AFD équivalent.

Exemple 3.3.1. Déterminisation d'un automate non déterministe :

Automate déterminisé :

Automate non déterministe :



Théorème 3.1. *Soit A un automate fini non déterministe avec des ε-transitions, alors il existe un automate fini non déterministe A', tel que $\mathcal{L}(A) = \mathcal{L}(A')$.*

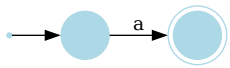
3.4 Équivalence entre expressions rationnelles et automates finis déterministes

Théorème 3.2 (Injection des expressions rationnelles vers les automates finis déterministes)**.** *Soit $r \in ERat$, alors il existe un automate fini déterministe $N(r)$ tel que $\mathcal{L}(A) = \mathcal{L}(r)$.*

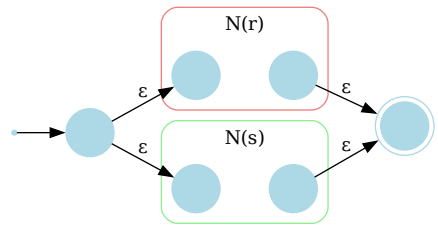
Démonstration. Nous allons construire un tel automate par induction sur la structure de r cependant la preuve du fait que cet automate reconnaît le langage associé à r est omise. Cette construction est appelée **construction de Thompson**.

$N(\varepsilon)$

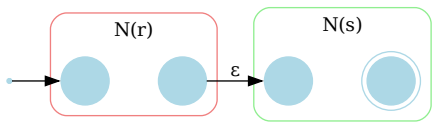
$N(\emptyset)$



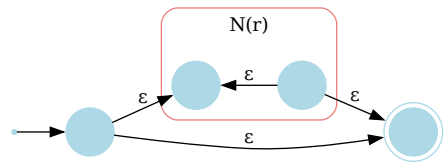
$N(a), a \in \Sigma$



$N(r+s)$



$N(rs)$



$N(r^*)$

En combinant ces constructions, on peut construire un automate fini non déterministe pour n'importe quelle expression rationnelle qui peut être determinisé pour obtenir un automate fini déterministe.

□

4 Minimisation d'automates

4.1 Morphismes d'automates

Définition 4.1 (Morphisme d'automates)**.** Soit $A = \langle Q, q_0, F, \delta \rangle$ et $A' = \langle Q', q'_0, F', \delta' \rangle$ sur un alphabet Σ. Alors un morphisme d'automates $\varphi : A \to A'$ est une fonction $\varphi : Q \to Q'$ telle que

1. $\varphi(q_0) = q'_0$
2. $\forall q \in Q, \quad \varphi(q) \in F' \iff q \in F$
3. $\forall q \in Q, \forall a \in \Sigma, \quad \delta'(\varphi(q), a) = \varphi(\delta(q, a))$ *i.e.* $\varphi \circ \delta_a = \delta'_a \circ \varphi$

$$\begin{tikzcd} Q \arrow[r, "\delta_a"] \arrow[d, "\varphi"'] & Q \arrow[d, "\varphi"] \\ Q' \arrow[r, "\delta_a"] & Q' \end{tikzcd}$$

Exercice 4.1.1. Soit $\varphi : A \to A'$ un morphisme d'automates déterministes, alors $\mathcal{L}(A) = \mathcal{L}(A')$

Démonstration.

$$\begin{aligned} w \in \mathcal{L}(A) &\iff \delta^*(q_0, w) \in F \\ &\iff \varphi(\delta^*(q_0, w)) \in F' \quad \text{par } 1 \\ &\iff \delta^*(\varphi(q_0), w) \in F' \quad \text{par } 3 \\ &\iff \delta^*(q_0', w) \in F' \quad \text{par } 2 \\ &\iff w \in \mathcal{L}(A') \end{aligned}$$

□

Définition 4.2 (Quotient d'un automate)**.** Soit $L \subseteq \Sigma^*$ et $w \in \Sigma^*$. Le quotient $w^{-1}L$ est défini par

$$w^{-1}L = \{u \in \Sigma^* \mid wu \in L\}$$

Définition 4.3. Soit $\langle Q, q_0, F, \delta \rangle$ sur l'alphabet Σ et $q \in Q$ alors on définit

$$L_q = \{w \in \Sigma^* \mid \delta^*(q, w) \in F\}$$

Lemme 4.1. *Soit $A = \langle Q, q_0, F, \delta \rangle$ un automate fini déterministe, $q \in Q$ et $w \in \Sigma^*$. Si $\delta^*(q_0, w) = q$ alors $L_q = w^{-1}\mathcal{L}(A)$*

Démonstration.

$$\begin{aligned} w \in L_q &\iff \delta^*(q, u) \in F \\ &\iff \delta^*(\delta^*(q_0, w), u) \in F \\ &\iff \delta^*(q_0, wu) \in F \\ &\iff wu \in \mathcal{L}(A) \\ &\iff u \in w^{-1}\mathcal{L}(A) \end{aligned}$$

□

Corollaire 4.1. *Si $L \subseteq \Sigma^*$ est un langage régulier alors l'ensemble $\left\{w^{-1}L \mid w \in \Sigma^*\right\}$ est fini.*

Démonstration. Si $L = \mathcal{L}(A)$, avec A $= \langle Q, q_0, F, \delta \rangle$, alors $\left|\left\{w^{-1}L \mid w \in \Sigma^*\right\}\right| \leq |Q|$. Car $\forall w \in \Sigma^*$, $w^{-1}L$ est le langage accepté par A à partir de l'état $\delta^*(q_0, w)$. □

Corollaire 4.2. *Si $\langle Q, I, F, \delta \rangle$ accepte un langage L alors $|Q| \geq \#$ quotients de L*

Définition 4.4 (Automate des quotients)**.** Soit $L \subseteq \Sigma^*$ un langage régulier, soit $Q = \left\{w^{-1}L \mid w \in \Sigma^*\right\}$ et $\langle Q, q_0, F, \delta \rangle$. Alors l'automate des quotients de L est défini par

- L'état initial est $\varepsilon^{-1}L$
- $F = \left\{w^{-1}L \mid w \in L\right\}$
- $\delta(w^{-1}L, a) = (wa)^{-1}L$

Remarque 4.1.1. *Si $w^{-1}L = (w')^{-1}L$, alors $(wa)^{-1}L = (w'a)^{-1}L$.*
Soit $w, w' \in \Sigma^*$ tel que $w^{-1}L = (w')^{-1}L$. Soit $a \in \Sigma$ et $u \in \Sigma^*$,

$$\begin{aligned} u \in (wa)^{-1}L &\iff wau \in L \\ &\iff au \in w^{-1}L \\ &\iff au \in w'^{-1}L \\ &\iff w'au \in L \\ &\iff (w'a)u \in L \\ &\iff u \in (w'a)^{-1}L \end{aligned}$$

Donc la fonction de transition δ de l'automate des quotients est bien définie.

Proposition 4.1. *Étant donné un automate A qui accepte le langage L, l'automate des quotients accepte aussi L et il est minimal parmi les automates acceptant L.*

Démonstration. $\delta^*(L, w) = w^{-1}L$ et donc $\underbrace{\delta^*(L, w) \in F}_{w \in \mathcal{L}(A)} \iff w \in L$

Et ainsi, par le corollaire 4.2, l'automate des quotients est minimal. □

Lemme 4.2. *Soit $L \subseteq \Sigma^*$ un langage régulier. Soit A_L l'automate des quotients de L et soit B un autre automate acceptant L. Soit reach(B) un sous-automate accessible de B. Alors on a un morphisme surjectif (un quotient) d'automates reach$(B) \twoheadrightarrow A_L$.*

$$\begin{tikzcd} & reach(B) \arrow[dl, two heads] \arrow[dr, hook] & \\ A_L & & B \end{tikzcd}$$

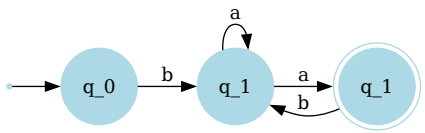
Si $B =< T, t_0, F, \delta >$ alors reach$(B) =< T', t_0, F', \delta' >$ où
— $T' = \{t \in T \mid \exists w \in \Sigma^*, \delta^*(t_0, w) = t\}$
— $F' = F \cap T'$
— $\delta' = \delta \cap (T' \times \Sigma \times T')$

Exercice 4.1.2. On définit $\varphi : \text{reach}(B) \to A$.
Si $t \in T', \exists w \in \Sigma^*$ tel que $t = \delta^*(t_0, w)$ On définit $\varphi(t) = w^{-1}L$
Montrer que φ est bien définie, *i.e.* , $\varphi t = w'^{-1}L \implies w^{-1}L = w'^{-1}L$.
Vérifions φ surjective. Si $w \in \Sigma^*$ et $w^{-1}L$ est un état de A_L alors $\varphi(\delta^*(t_0, w)) = w^{-1}L$
Si $|B| = |A_L|$, alors B est un automate accesible (Sinon $|A| \leq |\text{reach}(B)| < |B|$ ↯).
Donc $B = \text{reach}(B)$
De plus, le morphisme φ est une bijection et φ^{-1} est un morphisme d'automates. Donc $B \cong A_L$.

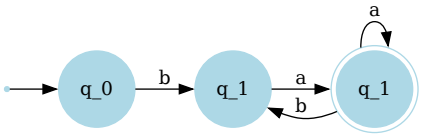
4.2 Minimisation d'automates

Rappel 4.2.1. *Pour tout langage rationnel L, il existe un unique (à isomorphisme près) automate déterministe minimal (avec le plus petit nombre possible d'états) qui reconnait L.*

Remarque 4.2.1. *C'est n'est pas vrai pour les automates non-déterministes.*



$(ba^+)^+$



$(ba^+)^+$

L'automate des résiduels de L est l'automate minimal reconnaissant L.

Nous allons voir trois algorithmes de minimisation différents :

— Algorithme de Brzozowski
— Algorithme de Moore
— Algorithme de Hopcroft

4.2.1 Algorithme de Brzozowski

Proposition 4.2. *Si A est un automate déterministe et accesible, i.e. , $\forall q \in Q, \exists$ un chemin $q_0 \to q$, et $A^\sim = det(mirr(A))$ est l'automate obtenu en déterminisant l'automate miroir de A. Alors $A^\sim$ est l'automate minimal qui reconnait $\widetilde{\mathcal{L}(A)}$.*

Démonstration. Soit $A = (Q, q_0, T)$, $L = \mathcal{L}(A)$. $\mathrm{mirr}(A) = (Q, T, q_0), M = \mathcal{L}(\mathrm{mirr}(A)) = \widetilde{L}$.

On note $X \cdot u$ et $\delta(X, u)$, X étant un ensemble d'états et u un mot, l'ensemble $\bigcup_{q \in X} \delta(q, a)$.

Nous allons montrer que $A^\sim$ est l'automate minimal pour M. Pour cela, il suffit de montrer que $A^\sim$ est isomorphe à l'automate des résiduels de M.

Pour cela, il faut montrer que si u et v sont deux mots quelconques tel que $u^{-1}M = v^{-1}M$, alors $T \cdot u = T \cdot v$ dans $A^\sim$.

Soit p un état de $T \cdot u$. Puisque A est accesible, il existe un chemin de $q_0 \to p$ dans A, et donc il existe un chemin $p \to q_0$ dans $\mathrm{mirr}(A)$. Soit w l'étiquette de ce chemin, alors le mot uw est l'étiquette d'un chemin réussi de $\mathrm{mirr}(A)$. En conséquence, $uw \in M$.

Mais $uw \in M \iff w \in u^{-1}M \iff w \in v^{-1}M \iff vw \in M$.

Donc vw est l'étiquette d'un chemin réussi Γ dans $\mathrm{mirr}(A)$. Étant donné que A est déterministe, tout chemin aboutissant dans q_0 et dont l'étiquette a w comme suffixe doit passer par p.

On en déduit que $p \in Tv$. □

Corollaire 4.3. *Soit A un automate, alors l'automate $det(mirr(det(mirr(A))))$ est l'automate minimal pour $\mathcal{L}(A)$.*

Démonstration. En effet,

$$\underbrace{\det(\mathrm{mirr}(\underbrace{\det(\mathrm{mirr}(A))}_{\text{Un automate déterministe et accesible qui reconnait } \widetilde{\mathcal{L}(A)}}))}_{\text{la proposition précédente garantit que cet automate est minimal pour } \widetilde{\widetilde{\mathcal{L}(A)}}=\mathcal{L}(A)}$$

□

Complexité. *La complexité de l'algorithme est en $O(2^n)$. En effet, la dernière détermination exécutée travaille sur un automate de taille au plus 2^n et produit un automate de taille 2^n. Sa complexité reste aussi en $O(2^n)$.*

Remarque 4.2.2. *Alors que d'autres algorithmes, tel que Moore, ont une complexité polynomiale, Brzozowski à différence des autres, ne nécessite pas que l'automate donné soit déterministe.*

4.2.2 Algorithme de Moore

Définition 4.5 (Congruence d'automates)**.** Soit $A = (Q, q_0, T, S)$ un automate déterministe et soit $\sim$ un relation d'équivalence définie sur l'ensemble Q. On dit que $\sim$ est une congruence si $\sim$ satisfait les conditions suivantes.

1. Compatibilité aves les transitions : Si $q \sim q'$ alors $\forall a \in \Sigma, \delta(q, a) \sim \delta(q', a)$
2. Saturation de A : Si $q \sim q'$ alors $q \in T \iff q' \in T$

Définition 4.6. Si $A = \langle Q, q_0, F, \delta \rangle$ est un AFD et $\sim$ une congruence définie dur Q. On définit l'automate quotient :

$$A/\sim = (Q', q_0', T', \delta')$$

avec
— $Q' = \{[q] \mid q \in Q\}$ (chaque état est étiqueté par une classe d'équivalence)
— $q_0' = [q_0]$
— $T' = \{[q] \mid q \in T\}$
— $\delta'([q], a) = [p]$ si et seulement si $q \in [qa]$

Proposition 4.3. *Si A est un automate déterministe et $\sim$ une congruence sur A, alors $\mathcal{L}(A) = \mathcal{L}(A/\sim)$*

Définition 4.7 (Congruence de Nerode)**.** Si $A = (Q, \{q_0\}, T, \delta)$ un AFD. $\forall q, q' \in Q$ on définit $q \cong q' \iff L_q = L_{q'}$.

Rappel 4.2.2. $L_q = \{w \mid \delta^*(q, w) \in T\}$. *Donc*

$$\begin{aligned} q \cong q' \iff L_q = L_{q'} &\iff \forall w, w \in L_q \text{ si et seulement si } w \in L_q \\ &\iff \forall w, \delta^*(q, w) \in T \text{ si et seulement si } \delta^*(q', w) \in T \end{aligned}$$

On en déduit que

$$q \ncong q' \iff \exists w, \delta^*(q, w) \in T \text{ et } \delta^*(q', w) \notin T \tag{1}$$

où $\delta^*(q, w) \notin T$ et $\delta^*(q,')\, w \in T$ sont deux états non équivalents, dits séparables.

Si q et q' sont séparables et w est un mot qui satisfait 1, on dira que w *sépare* q et q'.
Pour calculer la congruence de Nérode on introduit une famille de congruences :

$$\cong_i, i \in \mathbb{N}, q \cong_i q' \iff \forall w, |w| \leq i \implies \delta^*(q, w) \in T \iff \delta^*(q', w) \in T \tag{2}$$

Définition 4.8. Définition alternative
— $q \cong_0 q'$ si et seulement si $q \in T \iff q' \in T$
— $q \cong_{i+1} q'$ si et seulement si $q \cong_i q'$ et $\forall a \in \Sigma, \delta(q, a) \cong_i \delta(q', a)$

Remarque 4.2.3. *Par définition $\cong_{i+1}$ induit une partition de Q au moins aussi fine que celle induite par $\cong_i$.*

Les classes de $\cong_{i+1}$ sont obtenues en partitionnant des classes de $\cong_i$.

Remarque 4.2.4. *Puisque Q est fini, le processus de partitionnement doit se stabiliser. Autrement dit,*

$$\exists k \in \mathbb{N}^*, \textit{tel que } \cong_k = \cong_{k+j} \forall j \in \mathbb{N}^*$$

Proposition 4.4. *Si $k \in \mathbb{N}$ tel que $\cong_k = \cong_{k+j} \forall j \in \mathbb{N}$, alors $\cong_k = \cong$*

Démonstration. Soit $q, q' \in Q$, $q \cong_k q'$, montrer que $L_q = L_{q'}$. Exercice. □

Définition 4.9 (Algorithme de Moore)**.** L'algorithme consiste à créer un automate à partir des classes d'équivalence sur $\cong$. Les états sont les classes d'équivalence, on obtient les transitions en regardant le comportement d'un représentant de la classe et les états finaux sont les classes qui ont un représentant qui est un état final.

Complexité. *L'algorithme effectue n étapes et chaque étape dépense $O(n)$, donc sa complexité est en $O(n^2)$ (au pire).*

En fait, en moyenne $O(n \log n)$ (David, 2010) et meme $O(n \log \log n)$.

5 Monoïdes

Définition 5.1 (Monoïde)**.** Un monoïde est un tuple $(M, \times, 1)$ où M est un ensemble, $\times : M \times M \to M$ est une operation binaire sur M, $1 \in M$ tel que

1. $\times$ est une opération associative ($\forall x, y, z \in M, x \times (y \times z) = (x \times y) \times z$)
2. 1 est un élément neutre pout $\times$, c'est-à-dire , $\forall x \in M, 1 \times x = x \times 1 = x$

Exemple 5.0.1.
- $(\mathbb{N}, +, 0)$
- $(\Sigma^*, \cdot, \varepsilon)$
- $(\mathbb{Z}, +, 0)$ est un monoïde, mais aussi un groupe car tout élément possède un symétrique.
- Si Q est un ensemble, alors $(Q^Q, \circ, \mathrm{id})$

Définition 5.2 (Morphisme de monoïdes)**.** Soit $(M, \times_M, 1_M)$ et $(N, \times_N, 1_N)$ des monoïdes. Une fonction $h : M \to N$ est appelée un morphisme de monoïdes si et seulement si

1. $h(1_M) = 1_N$ (*h préserve l'élément neutre*)
2. $\forall x, y \in M, h(x \times_M y) = h(x) \times_N h(y)$ (*h préserve la multiplication*)

Proposition 5.1. *Soit Σ un ensemble et $(M, \cdot, 1)$ un monoïde. Soit $f : \Sigma \to M$ une fonction. Il existe un unique morphisme de monoïdes $\bar{f} : \Sigma^* \to M$ tel que $\bar{f}(a) = f(a), \forall a \in \Sigma$.*

$$\begin{tikzcd} \Sigma \arrow[r, hook] \arrow[d, "f"] & \Sigma^* \arrow[dl, dashed, "\exists! \bar{f}"] \\ M & \end{tikzcd}$$

Démonstration. — Existence :

Soit $w \in \Sigma^*$ de la forme $w = w_1 w_2 \dots w_n$ où $w_i \in \Sigma$.

Soit $\bar{f}(w) =^{def} f(w_1) \cdot \dots \cdot f(w_n)$ et $\bar{f}(\varepsilon) = 1_M$. Alors pour $a \in \Sigma, \bar{f}(a) = f(a)$.

Si $w = w_1 w_2 \dots w_n$ et $v = v_1 v_2 \dots v_m \in \Sigma^*$.

$$\begin{aligned}
\bar{f}(wv) = \bar{f}(w_1 w_2 \dots w_n v_1 v_2 \dots v_m) &= f(w_1) \cdot \dots \cdot f(w_n) \cdot f(v_1) \cdot \dots \cdot f(v_m) \\
&= (f(w_1) \cdot \dots \cdot f(w_n)) \cdot (f(v_1) \cdot \dots \cdot f(v_m)) \\
&= \bar{f}(w_1 w_2 \dots w_n) \cdot \bar{f}(v_1 v_2 \dots v_m) \\
&= \bar{f}(w) \cdot \bar{f}(v)
\end{aligned}$$

Donc $\forall w, v \in \Sigma^*, \bar{f}(wv) = \bar{f}(w)\bar{f}(v)$ et $\bar{f}(\varepsilon) = 1_M$ et ainsi, $\bar{f}$ est un morphisme de monoïdes.

— Unicité :

Si $h : \Sigma^* \to M$ est un morphisme de monoïdes tel que $h(a) = f(a), \forall a \in \Sigma^*$. Alors $h(\varepsilon) = 1_M$.

Comme h préserve la multiplication, on a $h(a_1 a_2 \dots a_n) = h(a_1) \cdots h(a_n) = f(a_1) \cdots f(a_n) = \bar{f}(a_1 a_2 \dots a_n)$.

Et donc $h = \bar{f}$

□

On dit que Σ^* est le monoïde librement engendré sur Σ

Définition 5.3 (Librement engendré)**.** H est librement engendré sur Σ si

$$\forall f : \Sigma \to M \text{ où } M \text{ est un monoïde}, \exists! \bar{f} : H \to M \text{ et } i : \Sigma \to H, \bar{f} \circ i = f$$

$H \cong \Sigma^*$

Exemple 5.0.2. TODO : Exemple

Remarque 5.0.1. $\mathbb{N} \cong \{1\}^*$

Définition 5.4 (Reconnaissance par monoïdes)**.** Soit Σ un alphabet fini. Soit $L \subseteq \Sigma^*$ un langage et soit $\varphi : \Sigma^* \to M$ un morphisme de monoïdes où $(M, \circ, 1_M)$ est un monoïde fini. On dit que φ reconnait L si et seulement si il existe $P \subseteq M$ tel que $L = \varphi^{-1}(P)$, c'est-à-dire $L = \{w \in \Sigma^* \mid \varphi(w) \in P\}$.

Exercice 5.0.1. φ reconnaît L si et seulement si $L = \varphi^{-1}(\varphi(L))$.

Proposition 5.2. *Soit $L \subseteq \Sigma^*$ un langage sur l'alphabet Σ. Alors L est reconnaissable si et seulement si L est reconnu par un morphisme de monoïdes $\varphi : \Sigma^* \to M$ où M est fini.*

Démonstration. — $\Rightarrow$

Soit $\varphi : \Sigma^* \to M$ un morphisme de monoïdes avec M fini qui reconnait le langage L. $\exists P \subseteq M, L = \varphi^{-1}(P)$.

Soit A l'automate suivant : $\langle M, 1_M, P, \delta \rangle$ où

$$\begin{aligned}
\delta : M \times \Sigma &\to M \\
(m, a) &\mapsto m \cdot \varphi(a)
\end{aligned}$$

On peut montrer par induction sur la longueur du mot que

$$\begin{array}{rcl} \delta^* : M \times \Sigma^* & \to & M \\ (1_M, w) & \mapsto & \varphi(w) \end{array}$$

si et seulement si

$$\begin{array}{rcl} A \text{ accepte le mot } w & \iff & \delta^*(1_M, w) \in P \iff \varphi(w) \in P \\ & \iff & w \in \varphi^{-1}(P) \iff w \in L. \end{array}$$

Donc A accepte le langage L

— $\Leftarrow$

Soit $A = \langle Q, q_0, F, \delta \rangle$ un automate déterministe complet qui accepte L. Soit

$$\begin{array}{rcl} \varphi : \Sigma^* & \to & Q^Q \quad \big((Q^Q, \circ, id_Q) \textit{ est bien un monoïde}\big) \\ \varphi(w)(q) & \mapsto & \delta^*(q, w) \end{array}$$

On a bien :

— Q^Q est un monoïde fini.

— $\varphi(\varepsilon)(q) = \delta^*(q, \varepsilon) = q$, donc $\begin{array}{rcl} \varphi(\varepsilon) : Q & \to & Q \\ q & \mapsto & q \end{array}$ est la fonction identité sur Q et donc φ preserve l'identité (élément neutre).

— $\varphi(w) \circ \varphi(w') = \varphi(ww'),\ \forall w, w' \in \Sigma^*$ car $\delta^*(\delta^*(q, w), w') = \delta^*(q, ww')$.

Soit $P \subseteq Q^Q$ défini par $P\{f : Q \to Q \mid f(q_0) \in F\}$.

$$\begin{array}{rcl} \varphi^{-1}(P) & = & \{w \in \Sigma^* \mid \varphi(w) \in P\} \\ & = & \{w \in \Sigma^* \mid \varphi(w)(q_0) \in F\} \\ & = & \{w \in \Sigma^* \mid \delta^*(q_0, w) \in F\} \\ & = & \{w \in \Sigma^* \mid w \text{ est accepté par } A\} \\ & = & L \end{array}$$

Donc L est reconnu par φ.

□

Définition 5.5. On va appeler $\varphi(\Sigma^*) \subseteq Q^Q$ le **monoïde de transition de l'automate** A.

Définition 5.6 (Congruence)**.** Soit $(M, \cdot, 1_M)$ un monoïde. Une congruence sur M est une relation d'équivalence $\sim \subseteq M \times M$ telle que

$$\forall m, m' \in M,\ m \sim m' \iff \forall n, r \in M, n \cdot m \cdot r \sim n \cdot m' \cdot r$$

Définition 5.7 (Congruence syntaxique d'un langage)**.** Soit $L \subseteq \Sigma^*$ un langage. Soit $\sim_L \subseteq \Sigma^* \times \Sigma^*$ la relation d'équivalence définie par :

$$w \sim_L w' \text{ si et seulement si } \forall u, v \in \Sigma^*, uwv \in L \iff uw'v \in L$$

Proposition 5.3. *$\sim_L$ est une congruence sur Σ^*.*

Démonstration. Exercice. □

Proposition 5.4. *Si $(M, \cdot, 1_M)$ est un monoïde et $\sim \subseteq M \times M$ est une congruence sur M, alors $M_{/\sim} = \{[m] \mid m \in M\}$, où $[m]$ note la classe d'équivalence de m par rapport à $\sim$.*

$M/\sim$ est un monoïde et $\begin{array}{rcl} h : M & \to & M/\sim \\ m & \mapsto & [m] \end{array}$ *est un morphisme de monoïdes.*

Démonstration. $(M/\sim, \cdot, [1_M])$ est un monoïde où $[m][m'] = [mm']$.

À montrer que cette multiplication est bien définie, c'est-à-dire si $m_2 \sim m$ et $m_2' \sim m'$ alors $m_2 m_2' \sim mm'$:

$$\begin{array}{rcll} m_2 & \sim & m & \\ m_2' & \sim & m' & \\ 1_M \cdot m_2 \cdot m_2' & \sim & 1_M \cdot m \cdot m_2' & (\textit{car} \sim \textit{est une congruence}) \\ m_2 \cdot m_2' & \sim & m \cdot m_2' & \\ \text{Mais } m \cdot m_2' & \sim & m \cdot m' & (\textit{car} \sim \textit{est une congruence}) \\ \implies m_2 \cdot m_2' & \sim & m \cdot m' & \end{array}$$

Donc la multiplication sur $M/\sim$ est bien définie.

$[1_M]$ est un élément neutre car $[m] \cdot [1_M] = [m \cdot 1_M] = [m]$ et pareil pour $[1_M] \cdot [m] = [m]$.

h est donc un morphisme de monoïdes :

— $h(1_M) = [1_M]$

— $h(m \cdot n) = [m \cdot n] = [m] \cdot [n] = h(m) \cdot h(n)$

□

Remarque 5.0.2. *h est un **morphisme surjectif** (aussi appelé un quotient).*

Définition 5.8. Soit $L \subseteq \Sigma^*$ un langage et soit $\sim_L$ la congruence syntaxique sur Σ^*. Le monoïde quotient $\Sigma^*/\sim_L$ est appelé le monoïde syntaxique de L.

Proposition 5.5. *Avec les notations ci-dessus, L est reconnu par le morphisme* $\begin{array}{rcl} \varphi : \Sigma^* & \to & \Sigma^*/\sim_L \\ w & \mapsto & [w] \end{array}$.

Remarque 5.0.3. *Si $w \in L$ et $w' \sim_L w$ alors $w' \in L$, parce que $w' \sim_L w$.*

Comme $\varepsilon w \varepsilon \in L$, $\varepsilon w' \varepsilon$ donc $w' \in L$.

Démonstration. Soit $P \subseteq \Sigma^*/\sim_L$ donné par $P = \{[w] \mid w \in L\}$.

On doit montrer que $L = \varphi^{-1}(P)$, c'est-à-dire , $L = \{w \in \Sigma^* \mid \varphi(w) \in P\}$

En utilisant la remarque 5.0.3 on a que si $w \in L$ alors $[w] \subseteq L$.

On en conclut que

$$L = \bigcup_{w \in L} [w] \iff L = \varphi^{-1}(P)$$

□

Proposition 5.6. *Le monoïde syntaxique a une propriété similaire à 4.2 : $\forall (M, \cdot, 1_M)$ monoïde qui reconnait un langage L, on a le diagramme suivant :*

$$\begin{tikzcd} & (N, \cdot, 1_N) \arrow[dl, two heads] \arrow[dr] & \\ (\Sigma^*/\sim_L, \cdot, [\varepsilon]) & & (M, \cdot, 1_M) \end{tikzcd}$$

Terminologie 5.1. *On dit que le monoïde syntaxique de L divise tout monoïde qui accepte L.*

Lemme 5.1. *Si N divise M et N reconnait un langage L alors M reconnait L.*

Démonstration.

$$\begin{tikzcd} & T \arrow[dl, two heads] \arrow[dr, hook] & \\ N & & M \end{tikzcd}$$

$\varphi : \Sigma^* \to N$ et $P \subseteq N$ tel que $L = \varphi^{-1}(P)$

Exercice : montrer que on peut construire $\varphi'' : \Sigma^* \to M$ qui reconnait L. □

Proposition 5.7. *Un monoïde M reconnait un langage L si et seulement si le monoïde syntaxique $\Sigma^*/\sim_L$ divise M.*

Démonstration.

— $\Leftarrow$

$\Sigma^*/\sim_L$ reconnait L. Par le lemme, si $\Sigma^*/\sim_L$ divise M, alors M reconnait L.

— $\Rightarrow$

Supposons que M reconnait L. Donc on a $\varphi : \Sigma^* \to M$ et $P \subseteq M$ avec $\varphi^{-1}(P) = L$.
Soit $N = \varphi(\Sigma^*)$ l'image directe de Σ^* par φ.
Alors $N \hookrightarrow M$ est un sous-monoïde de M.
Je voudrais avoir un quotient $h : N \twoheadrightarrow \Sigma^*/\sim$.
Si $n \in N$ alors $\exists w \in \Sigma^*$ tel que $\varphi(w) = n$. On définit $h(n) = [w]$.
Il faut montrer que

1. h est bien défini (si $\varphi(w') = \varphi(w) \implies [w] = [w']$).
2. h est un morphisme de monoïdes.
3. h est surjectif.

Montrons cela :

— 1 : Supposons que $\varphi(w) = \varphi(w')$.
On veut montrer que $[w] = [w']$ c'est-à-dire $w \sim_L w'$, c'est-à-dire $\forall u, v \in \Sigma^*, uwv \in L \iff uw'v \in L$.
Soient $u, v \in \Sigma^*$.

$$\begin{aligned} uwv \in L &\iff \varphi(uwv) \in P \\ &\iff \varphi(u)\varphi(w)\varphi(v) \in P \quad (\textit{car } \varphi \textit{ est un morphisme}) \\ &\iff \varphi(u)\varphi(w')\varphi(v) \in P \quad (\textit{car } \varphi(w) = \varphi(w')\ ) \\ &\iff \varphi(uw'v) \in P \\ &\iff uw'v \in L \end{aligned}$$

Donc $w \sim_L w'$.

Exercice : finir la preuve.

□